

## 7.1 Introdução

Os capítulos anteriores trataram, em geral, de estruturas genéricas, adequadas à representação de quaisquer massas de dados. Já no [Capítulo 6](#), algo mais específico foi apresentado: uma estrutura eficiente para modelar dados de entrada de um determinado problema, a manipulação de elementos de uma tabela com prioridades. O capítulo presente é diferente; aqui serão descritas técnicas distintas para solucionar um problema que aparece como pré-processamento em muitas aplicações que envolvam o uso de tabelas (isto é, praticamente todas!) – a obtenção de uma tabela ordenada.

O problema da ordenação foi um dos primeiros a gerar discussões sobre implementações eficientes ou não. É razoavelmente fácil comparar dois algoritmos quando aplicados a uma mesma massa de dados. Entretanto, massas de dados modelam situações e, muitas vezes, devido a particularidades das mesmas, alguns métodos se comportam melhor que outros, independentemente de sua avaliação em testes genéricos. O melhor exemplo disso são os métodos mais simples, como a ordenação bolha e a ordenação por inserção. Apesar de a complexidade de pior caso ser ruim, ambos são bastante utilizados em razão de sua extrema simplicidade de implementação. Também não pode ser esquecido que esses métodos podem ser convenientes quando a tabela é pequena ou está quase ordenada.

O objetivo deste capítulo é o estudo de algoritmos de ordenação, a avaliação de suas complexidades e implementações mais eficientes. Os primeiros métodos apresentados, a *ordenação bolha* e a *ordenação por inserção*, vistos nas [Seções 7.2](#) e [7.3](#), são métodos bastante simples e de fácil implementação. Da simplicidade dos mesmos decorre a complexidade de  $O(n^2)$ . Na [Seção 7.4](#) será vista a *ordenação por intercalação*, método bastante interessante que utiliza o procedimento de intercalação de listas visto no [Exercício 2.14](#). Os mais conhecidos (e muito eficientes) *quicksort* (ordenação rápida) e *heapsort* (ordenação em heap) são vistos nas [Seções 7.5](#) e [7.6](#). A [Seção 7.7](#) é dedicada a estabelecer um limite inferior para a complexidade de algoritmos de ordenação.

## 7.2 Ordenação Bolha (*Bubble Sort*)

O método de ordenação bolha é bastante simples, e talvez seja o método de ordenação mais difundido. Uma iteração do mesmo se limita a percorrer a tabela do início ao fim, sem interrupção, trocando de posição dois elementos consecutivos sempre que estes se apresentem fora de ordem. Intuitivamente percebe-se que a intenção do método é mover os elementos maiores em direção ao fim da tabela. Ao terminar a primeira iteração pode-se garantir que as trocas realizadas posicionam o maior elemento na última posição. Na segunda iteração, o segundo maior elemento é posicionado, e assim sucessivamente. O processamento é repetido então  $n - 1$  vezes. O algoritmo que se segue implementa este método. A tabela se encontra armazenada na estrutura L. O algoritmo ordena L segundo valores não decrescentes do campo *chave*.

### ■ Algoritmo 7.1 Ordenação bolha de uma tabela com n elementos

```
para i = 1, ... n faça
    para j = 1, ... n - 1 faça
        se L[j].chave > L[j + 1].chave então
            trocar(L[j], L[j + 1])
```

O [Algoritmo 7.1](#) é claramente ruim. Sua complexidade de pior caso é igual à de melhor caso,  $O(n^2)$ , devido aos percursos estipulados para as variáveis *i* e *j*. Pode-se, entretanto, pensar em alguns critérios de parada que levariam em consideração comparações desnecessárias, isto é, comparações executadas em partes da tabela sabidamente já ordenadas.

O primeiro critério de parada resulta do fato de que, se a tabela está ordenada, o algoritmo é executado em toda sua extensão desnecessariamente. Uma variável lógica *mudou* é então introduzida com a finalidade de sinalizar se pelo menos uma troca foi realizada. Caso isso não ocorra, o algoritmo pode ser encerrado. Essa simples alteração afeta a complexidade do melhor caso do algoritmo, que passa a ser  $O(n)$ , uma vez que, se a tabela já está ordenada, apenas um percurso é realizado.

A segunda modificação decorre do próprio método: como, a cada passo, um elemento é garantidamente transportado para o fim da tabela, o número de elementos da mesma pode ser diminuído. Na realidade, a posição da última troca (armazenada, no algoritmo, na variável *guarda*) indica que todos os elementos posteriores já estão ordenados. O algoritmo pode então utilizar esta posição para atualizar o limite superior da tabela, que inicialmente é o próprio número de elementos.

■ **Algoritmo 7.2** Ordenação bolha com critério de parada

```
mudou := V; n' := n; guarda := n
enquanto mudou faça
    j := 1; mudou := F
    enquanto j < n' faça
        se L[j] . chave > L[j + 1] . chave então
            trocar(L[j], L[j + 1])
            mudou := V
            guarda := j
            j := j + 1
    n' := guarda
```

A [Figura 7.1](#) apresenta um exemplo de aplicação do [Algoritmo 7.2](#), sendo quatro iterações suficientes para a ordenação desejada. A última coluna indica o número de trocas realizadas durante a iteração considerada.

O [Algoritmo 7.2](#) é bem mais eficiente; sua complexidade de pior caso, entretanto, permanece  $O(n^2)$ . A entrada que justifica esse resultado é a tabela ordenada em ordem inversa.

| Iteração         | Tabela               | Trocas |
|------------------|----------------------|--------|
| tabela inicial   | 40 37 95 42 39 51 60 |        |
| após 1ª iteração | 37 40 42 39 51 60 95 | 5      |
| após 2ª iteração | 37 40 39 42 51 60 95 | 1      |
| após 3ª iteração | 37 39 40 42 51 60 95 | 1      |
| após 4ª iteração | 37 39 40 42 51 60 95 | 0      |

FIGURA 7.1 Um exemplo de ordenação bolha.

### 7.3 Ordenação por Inserção

O método de ordenação por inserção é também bastante simples, sendo sua complexidade equivalente à da ordenação bolha. Imagine uma tabela já ordenada até o *i*-ésimo elemento. A ordenação da tabela pode ser estendida até o (*i* + 1)-ésimo elemento por meio de comparações sucessivas deste com os elementos anteriores, isto é, com o *i*-ésimo elemento, com o (*i* – 1)-ésimo elemento etc., procurando sua posição correta na parte da tabela que já está ordenada. Pode-se então deduzir um algoritmo para implementar o método: considera-se sucessivamente todos os elementos, a partir do segundo deles, em relação à parte da tabela formada pelos elementos anteriores ao elemento considerado em cada iteração.

| Iteração          | Tabela               | Trocas |
|-------------------|----------------------|--------|
| tabela inicial    | 40 37 95 42 23 51 27 |        |
| após <i>i</i> = 2 | 37 40 95 42 23 51 27 | 1      |
| após <i>i</i> = 3 | 37 40 95 42 23 51 27 | 0      |
| após <i>i</i> = 4 | 37 40 42 95 23 51 27 | 1      |
| após <i>i</i> = 5 | 23 37 40 42 95 51 27 | 4      |
| após <i>i</i> = 6 | 23 37 40 42 51 95 27 | 1      |
| após <i>i</i> = 7 | 23 27 37 40 42 51 95 | 5      |

FIGURA 7.2 Um exemplo de ordenação por inserção.

A [Figura 7.2](#) mostra um exemplo da execução do algoritmo. A tabela é reapresentada após a execução do posicionamento de cada elemento, a partir do segundo.

O [Algoritmo 7.3](#) apresenta a ordenação por inserção da tabela  $L$ , de  $n$  elementos, segundo o seu campo *chave*. Para evitar erros de implementação, a tabela deve ser acrescida de uma posição  $L[0]$ , que pode receber em seu campo *chave* qualquer valor. Esse valor será testado no caso em que a posição definitiva do elemento que está sendo analisado seja a primeira. Essa comparação, entretanto, não tem efeito, uma vez que, nesse caso,  $j < 1$  e o teste resulta falso.

■ **Algoritmo 7.3** Ordenação por inserção de uma tabela com  $n$  elementos

```
para  $i = 2, \dots, n$  faça
     $prov := L[i]; valor := L[i].chave$ 
     $j := i - 1$ 
    enquanto  $j \geq 1$  e  $valor < L[j].chave$  faça
         $L[j + 1] := L[j]$ 
         $j := j - 1$ 
     $L[j + 1] := prov$ 
```

A análise da complexidade do algoritmo é bastante simples e utiliza o conceito de inversões. Dados dois elementos da tabela  $L[i]$  e  $L[j]$ , sendo  $i < j$ , uma inversão ocorre quando  $L[i].chave > L[j].chave$ . Como pode ser visto no exemplo apresentado na [Figura 7.2](#), o número de trocas realizadas pelo algoritmo é exatamente o número de inversões da tabela. O percurso em cada iteração termina exatamente quando a inversão não ocorre. Então, o algoritmo tem complexidade de melhor caso  $O(n)$ , que ocorre quando o número de inversões é zero. No pior caso, quando a tabela está em ordem inversa, são executadas  $n - 1$  iterações (para  $i = 2, \dots, n$ ) e, em cada uma delas, removidas  $i - 1$  inversões. Logo,

$$\sum_{i=1}^{n-1} Inv(i) = 1 + 2 + \dots + (n - 1) = O(n^2).$$

## 7.4 Ordenação por Intercalação (Mergesort)

O método apresentado nesta seção tem uma característica interessante, pois tem como procedimento básico o procedimento de intercalação de listas, mencionado no [Exercício 2.14](#).

A ideia básica do método é intercalar as duas metades da lista desejada quando estas já se encontram ordenadas. Na realidade, deseja-se então ordenar primeiramente as duas metades, o que pode ser feito utilizando recursivamente o mesmo conceito.

Primeiramente, o processo de intercalação será revisto. Sejam duas listas  $A$  e  $B$ , ordenadas, com respectivamente  $n$  e  $m$  elementos. As duas listas são percorridas por ponteiros  $ptA$  e  $ptB$ , armazenando o resultado da intercalação na lista  $C$ , apontada pelo ponteiro  $ptC$ . O primeiro elemento de  $A$  é comparado com o primeiro elemento de  $B$ ; o menor valor é colocado em  $C$ . O ponteiro da lista onde se encontra o menor valor é incrementado, assim como o ponteiro da lista resultado; o processo se repete até que uma das listas seja esgotada. Neste ponto, os elementos restantes da outra lista são copiados na lista resultado. A [Figura 7.3](#) ilustra o processo.

Seja  $L$  a lista que se deseja ordenar. O método de ordenação por intercalação consiste em dividir a lista original em duas metades e ordená-las; o resultado são duas listas ordenadas que podem ser intercaladas. Para ordenar cada uma das metades o processo considerado é o mesmo, sendo o problema dividido em problemas menores, que são sucessivamente solucionados. O [Algoritmo 7.4](#) apresenta a solução, incluindo o procedimento de intercalação específico para dois trechos da mesma tabela, processo semelhante ao exibido na [Figura 7.3](#) para tabelas distintas. Observe que, também neste caso, é necessário utilizar uma lista temporária para armazenar o resultado da intercalação. Para evitar a criação de uma nova área de trabalho a cada chamada do procedimento, essa área temporária é considerada como dado do problema. O [Algoritmo 7.4](#) apresenta a implementação do

método. Após a análise do algoritmo será explicado como essa tabela temporária pode ser utilizada para uma implementação ainda mais eficiente.

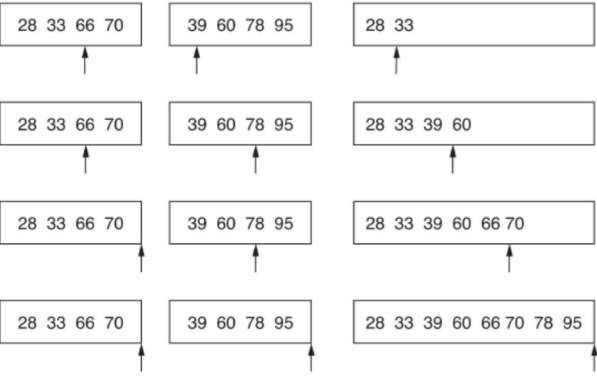


FIGURA 7.3 O processo de intercalação.

■

**Algoritmo 7.4** Ordenação por intercalação de uma tabela com n elementos

```

procedimento mergesort (esq, dir);
    se esq < dir então
        centro := ⌊ (esq + dir)/2 ⌋ ;
        mergesort(esq, centro);
        mergesort(centro + 1, dir);
        intercalar(esq, centro + 1, dir);
procedimento intercalar(L, Tmp, ini1, ini2, fim2);
    fim1 := ini2 – 1; nro := 0
    ind := ini1
    enquanto (ini1 ≤ fim1) e (ini2 ≤ fim2) faça
        se L[ini1] . chave se L[ini1] L[ini2] . chave então
            Tmp[ind] := L[ini1]
            ini1 := ini1 + 1
        senão Tmp[ind] := L[ini2]
            ini2 := ini2 + 1
        ind := ind + 1; nro := nro + 1
    enquanto {ini1 ≤ fim1} faça
        Tmp[ind] := L[ini1]
        ini1 := ini1 + 1; ind := ind + 1; nro := nro + 1
    enquanto {ini2 ≤ fim2} faça
        Tmp[ind] := L[ini2]
        ini2 := ini2 + 1; ind := ind + 1; nro := nro + 1
    para i := 1, ..., nro faça
        L[i + ini1 – 1] := Tmp[i + ini1 – 1]
mergesort (1, n)
    
```

### 7.4.1 Análise do Algoritmo

O tempo de execução da ordenação por intercalação sobre uma tabela com  $n$  elementos,  $T(n)$ , é o resultado da soma dos tempos de execução das duas chamadas recursivas, cada uma delas com tempo de execução  $T(n/2)$ . Como condição inicial, para  $n = 1$ ,  $T(1) = 1$ . Assumindo-se que  $n$  é uma potência de 2, a fórmula básica que expressa o número de iterações executadas é:

$$T(n) = 2T(n/2) + n. \quad (7.1)$$

Para determinar o valor desejado, primeiramente ambos os membros são divididos por  $n$ :

$$\frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1. \quad (7.2)$$

Como a equação é válida para qualquer potência de 2, pode-se escrever:

$$\frac{T(n/2)}{n/2} = \frac{T(n/4)}{n/4} + 1. \quad (7.3)$$

$$\frac{T(n/4)}{n/4} = \frac{T(n/8)}{n/8} + 1. \quad (7.4)$$

$$\frac{T(n/2)}{2} = \frac{T(1)}{1} + 1. \quad (7.5)$$

A partir da [Equação \(7.2\)](#), excetuando-se a última, a primeira parcela do lado direito da igualdade é igual ao termo do lado esquerdo da equação seguinte. A soma das equações resulta em:

$$\frac{T(n)}{n} = \frac{T(1)}{1} + \log n. \quad (7.6)$$

Concluindo-se que

$$T(n) = n + n \log n. \quad (7.7)$$

A complexidade de pior caso do método é, então,  $O(n \log n)$ . Entretanto, mesmo com este bom resultado a ordenação por intercalação não é um dos métodos de ordenação mais empregados. Sua eficiência depende da cuidadosa implementação da tabela temporária. A primeira providência necessária já foi tomada: utilizar somente uma área de trabalho ao longo de todo o processo. A segunda seria evitar a cópia executada após cada intercalação, o que pode ser feito alternando-se as tabelas  $L$  e  $Tmp$  em níveis distintos da recursão.

## 7.5 Ordenação Rápida (*Quicksort*)

O nome *quicksort* (ordenação rápida) já indica o que se deve esperar do método, que é, na realidade, um dos mais eficientes dentre os conhecidos. Dada uma tabela  $L$  com  $n$  elementos, o procedimento recursivo para ordenar  $L$  consiste nos seguintes passos:

- se  $n = 0$  ou  $n = 1$  então a tabela está ordenada;
- escolha qualquer elemento  $x$  em  $L$  – este elemento é chamado pivô;
- separe  $L - \{x\}$  em dois conjuntos de elementos disjuntos:  $S_1 = \{w \in L - \{x\} | w < x\}$  e  $S_2 = \{w \in L - \{x\} | w \geq x\}$ ; o procedimento de ordenação é chamado recursivamente para  $S_1$  e  $S_2$ ;
- $L$  recebe a concatenação de  $S_1$ , seguido de  $x$ , seguido de  $S_2$ .

Dois pontos são decisivos para o bom desempenho do algoritmo: a escolha do pivô e o particionamento da tabela. Esses pontos são analisados a seguir.

A escolha do pivô mais utilizada é tomar o primeiro elemento como tal. Isto é aceitável se a tabela é reconhecidamente aleatória. Se, entretanto, a tabela está ordenada na ordem inversa à desejada, esta escolha provoca o pior desempenho do algoritmo. A partição obtida nesse caso separa a tabela de tal forma que o conjunto  $S_1$  contém  $n - 1$  elementos e o conjunto  $S_2$  é vazio. Sendo assim, o primeiro elemento não é uma boa escolha para pivô.

Uma estratégia alternativa é a escolha aleatória do pivô, mas a geração de números aleatórios poderia pesar no tempo de execução. A situação ideal seria a escolha de um pivô tal que proporcionasse a partição da tabela em dois subconjuntos de dimensão equivalente. Para isso, a mediana da tabela deveria ser o elemento selecionado, lembrando que a mediana de um

conjunto de  $n$  elementos é o  $n/2$ -ésimo maior número. Infelizmente, o cálculo desse elemento seria também muito custoso. Uma solução utilizada com bons resultados é a escolha da mediana dentre três elementos: o primeiro, o último e o central.

Solucionada a escolha do pivô, passa-se ao particionamento. Na descrição do algoritmo, os elementos iguais ao pivô foram colocados no conjunto  $S_2$ . Mas essa decisão, geralmente, depende da implementação. Antes disso, porém, observe o funcionamento do algoritmo considerando todos os elementos distintos.

Primeiramente, o pivô é afastado da tabela a ser percorrida; isto pode ser feito colocando-o na última posição e considerando somente o restante da tabela. Em seguida, dois ponteiros são utilizados. O primeiro,  $i$ , é inicializado apontando para o primeiro elemento da tabela, percorrendo-a enquanto os valores apontados são menores do que o pivô. O segundo,  $j$ , é inicializado na penúltima posição, efetuando a tarefa inversa. Os percursos são interrompidos quando  $i$  aponta para um elemento maior do que o pivô e  $j$  aponta para um elemento menor do que o pivô. Duas situações podem ocorrer. Se  $i < j$ , os elementos da tabela devem ser trocados e o procedimento pode prosseguir. Se  $i > j$ , a partição já está determinada. O pivô, que se encontra na última posição da tabela, deve ser trocado com o elemento de índice  $i$ . Após a troca, os elementos de índice menor do que  $i$  são menores do que o pivô; o índice  $i$  indica a posição definitiva do pivô, e os de índice maior do que  $i$  formam o conjunto de elementos maiores do que o pivô. Note que o elemento de índice  $i$  foi trocado com o elemento que está na última posição da tabela (cuja chave é o pivô), ficando então na parte correta.

A [Figura 7.4](#) mostra uma passagem da execução do algoritmo no exemplo já utilizado anteriormente, na [Figura 7.2](#). Primeiramente o pivô é escolhido entre os valores 40, 42 e 27. O pivô determinado, 40, é trocado com a última posição e o percurso sobre a tabela com os índices  $i$  e  $j$  é iniciado. No momento em que esse percurso não pode mais prosseguir,  $i$  aponta para 95 e  $j$ , para 23. Como  $i < j$ , os dois elementos são trocados, o percurso é reiniciado e para novamente quando  $i$  aponta para 42 e  $j$ , para 23. Agora, entretanto,  $i > j$  e a posição do pivô está decidida. O último elemento ocupa sua posição definitiva (a posição apontada por  $i$ ), e os dois subconjuntos estão determinados: o de elementos menores do que o pivô,  $\{27, 37, 23\}$ , e o de maiores,  $\{95, 51, 42\}$ . O algoritmo prossegue, sendo aplicado a cada um dos subconjuntos.

Agora analisa-se a questão das chaves repetidas. Observe que, se elas acontecem na tabela L, o particionamento será afetado pelo teste executado no percurso dos ponteiros. A escolha do teste mais conveniente pode ser feita utilizando-se como exemplo uma tabela em que todas as chaves são iguais. Se os dois testes executados quando do percurso dos índices  $i$  e  $j$  ignoram chaves iguais ao pivô, haverá necessidade do acréscimo de um critério de parada, e mais, o pivô será colocado na extremidade da tabela, o que constitui o pior particionamento possível. Por outro lado, se nos dois testes o percurso for suspenso quando a chave é igual ao pivô, trocas desnecessárias serão feitas, mas o particionamento da tabela ocorrerá de forma mais equilibrada.

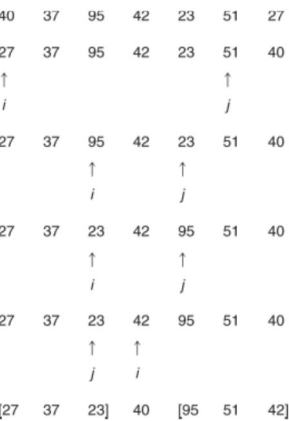


FIGURA 7.4 Ordenação rápida.

O [Algoritmo 7.5](#) apresenta a ordenação rápida da tabela L, de  $n$  elementos, segundo o seu *campo chave*. As variáveis *ini* e *sup*, parâmetros do procedimento, são índices que indicam o primeiro e o último elemento da parte tabela que está sendo processada na chamada. O procedimento *PIVO*, chamado no decorrer da ordenação, escolhe a mediana de três elementos.

■ **Algoritmo 7.5** Ordenação rápida de uma tabela com  $n$  elementos

**procedimento** *quicksort* (*ini*, *fim*)

**se** *fim* – *ini* < 2 **então**

**se** *fim* – *ini* = 1 **então**

**se**  $L[ini] \cdot chave > L[fim] \cdot chave$  **então**

*trocar*( $L[ini]$ ,  $L[fim]$ );

**senão** *PIVO*(*ini*, *fim*, *mediana*)

*trocar*( $L[mediana]$ ,  $L[fim]$ )

*i* := *ini*; *j* := *fim* – 1

*key* :=  $L[fim] \cdot chave$

**enquanto** *j* ≥ *i* **faça**

**enquanto**  $L[i] \cdot chave < key$  **faça**

*i* := *i* + 1

**enquanto**  $L[j] \cdot chave > key$  **faça**

*j* := *j* – 1

**se** *j* ≥ *i* **então**

*trocar*( $L[i]$ ,  $L[j]$ )

*i* := *i* + 1; *j* := *j* – 1

*trocar*( $L[i]$ ,  $L[fim]$ )

*quicksort*(*ini*, *i* – 1)

*quicksort*(*i* + 1, *fim*);

*quicksort*(1, *n*)

Uma observação deve ser feita a respeito da implementação do método. Um único critério de parada é utilizado, o teste  $j \geq i$ , independente, portanto, do tamanho da tabela. Isto é possível porque, após cada troca, o índice *i* é incrementado e *j* é decrementado.

### 7.5.1 Análise do Algoritmo

A análise do tempo despendido pela ordenação rápida é bastante semelhante à análise vista na ordenação por intercalação. O tempo de execução do método rápido sobre uma tabela com *n* elementos,  $T(n)$ , é o resultado da soma dos tempos de execução das duas chamadas recursivas com o tempo linear despendido no particionamento da tabela. A análise do algoritmo será feita assumindo-se que o pivô é escolhido aleatoriamente. Assume-se também que  $T(0) = T(1) = 1$ . A fórmula básica que expressa o número de iterações executadas é:

$$T(n) = T(i) + T(n - i - 1) + c n \quad (7.8)$$

onde *i* é o número de elementos em  $S_i$  e *c* é uma constante. Serão estudados três casos: o pior, o melhor e o caso médio.

#### Análise do pior caso

No pior caso, o pivô é o menor elemento, o que faz com que, no momento em que a tabela é partida, um dos subconjuntos fique vazio. Assim, *i* = 0 e, como  $T(0) = 1$  é insignificante, pode ser ignorado. A fórmula básica de medida do tempo de execução é:

$$T(n) = T(n - 1) + c n, n > 1 \quad (7.9)$$

A [Equação \(7.9\)](#) pode ser usada repetidamente para tabelas cada vez menores, considerando-se sempre que o pivô é o menor elemento. Tem-se então:

$$T(n - 1) = T(n - 2) + c(n - 1) \quad (7.10)$$

$$T(n-2) = T(n-3) + c(n-2) \quad (7.11)$$

⋮

$$T(2) = T(1) + c(2) \quad (7.12)$$

Somando-se as equações acima, a partir da [Equação \(7.9\)](#), obtém-se:

$$T(n) = T(1) + c \sum_{i=2}^n i = O(n^2) \quad (7.13)$$

Este resultado mostra a necessidade da escolha criteriosa do pivô.

## Análise do melhor caso

O melhor caso ocorre quando o pivô é a mediana de uma tabela sem elementos repetidos. Assim os dois subconjuntos determinados,  $S_1$  e  $S_2$ , têm a mesma cardinalidade. Este equilíbrio gera mais subconjuntos efetivamente tratados. Tem-se então:

$$T(n) = 2T(n/2) + cn \quad (7.14)$$

Dividindo ambos os membros da [Equação \(7.14\)](#) por  $n$ :

$$\frac{T(n)}{n} = \frac{T(n/2)}{n/2} + c \quad (7.15)$$

Usando repetidamente a [Equação \(7.15\)](#):

$$\frac{T(n/2)}{n/2} = \frac{T(n/4)}{n/4} + c \quad (7.16)$$

⋮

$$\frac{T(2)}{1} = \frac{T(1)}{1} + c \quad (7.17)$$

Adicionando as equações acima, a partir da [Equação \(7.15\)](#), resulta:

$$\frac{T(n)}{n} = \frac{T(1)}{1} + c \log n \quad (7.18)$$

A [Equação \(7.18\)](#) permite dizer que

$$T(n) = cn \log n + n = O(n \log n). \quad (7.19)$$

## Análise do caso médio

Assume-se que cada possível dimensão do conjunto  $S$  tem igual probabilidade, logo  $1/n$ . O valor médio de  $T(i)$ , e também de  $T(n-i-1)$ , é

A [Equação \(7.8\)](#) fica então:

$$T(n) = \frac{2}{n} \left[ \sum_{j=0}^{n-1} T(j) \right] + cn \quad (7.20)$$

Multiplicando a [Equação \(7.20\)](#) por  $n$  tem-se:

$$nT(n) = 2 \left[ \sum_{j=0}^{n-1} T(j) \right] + cn^2 \quad (7.21)$$

$$(n-1)T(n-1) = 2 \left[ \sum_{j=0}^{n-2} T(j) \right] + c(n-1)^2. \quad (7.22)$$

Efetuada a diferença entre as [Equações \(7.21\) e \(7.22\)](#):

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2cn - c \quad (7.23)$$

Abandonando, na direita da equação, a constante  $c$ :

$$nT(n) = (n+1)T(n-1) + 2cn \quad (7.24)$$

Tem-se agora a fórmula de  $T(n)$  em função de  $T(n-1)$ . Dividindo então por  $n(n+1)$  e repetindo sucessivamente:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2c}{n+1} \quad (7.25)$$

$$\frac{T(n-1)}{n} = \frac{T(n-2)}{n-1} + \frac{2c}{n} \quad (7.26)$$

$$\frac{T(n-2)}{n-1} = \frac{T(n-2)}{n-2} + \frac{2c}{n-1} \quad (7.27)$$

⋮

$$\frac{T(2)}{3} = \frac{T(1)}{2} + \frac{2c}{3} \quad (7.28)$$

Adicionando-se as equações a partir de [\(7.25\)](#), tem-se

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + 2c \sum_{i=3}^{n+1} \frac{1}{i}. \quad (7.29)$$

O somatório que aparece na [Equação \(7.29\)](#) pode ser calculado com auxílio do parâmetro  $\gamma$ , conhecido como constante de Euler-Mascheroni. Sabe-se que

$$\gamma = \lim \left( \sum_{k=1}^n \frac{1}{k} - \ln n \right).$$

O somatório é então aproximadamente igual a  $O(\log n)$ . Logo,

$$\frac{T(n)}{n+1} = O(\log n).$$

E, finalmente,  $T(n) = O(n \log n)$ .

## 7.6 Ordenação em Heap (*Heapsort*)

No [Capítulo 6](#) foi visto o conceito de lista de prioridades e sua implementação mais eficiente, utilizando um heap. Uma aplicação conhecida dessa estrutura é um algoritmo que efetua a ordenação de uma tabela. A implementação deste método utiliza a estrutura de dados heap e os procedimentos que a manipulam.

Seja  $L$  a tabela a ser disposta em ordem não decrescente segundo as suas chaves, sendo  $L$  uma lista de prioridades. A prioridade atribuída a cada nó é considerada como sendo igual ao valor de sua chave de ordenação. Sabe-se que o primeiro elemento da lista é o de maior prioridade. Esse elemento é obrigatoriamente o último na lista ordenada. Pode-se então realizar a

troca do primeiro com o último elemento e diminuir a tabela. A tabela L, diminuída de um elemento, pode não ser uma lista de prioridades (o seu primeiro elemento foi modificado) e deve ser reorganizada. Repetindo-se este processo para todos os elementos de L, o resultado final é a tabela ordenada.

O [Algoritmo 7.6](#) apresenta a ordenação em heap da tabela L, de  $n$  elementos, segundo o seu *campo chave*. Primeiramente, o heap deve ser construído utilizando-se o procedimento *arranjar* (ver [Algoritmo 6.6](#) no [Capítulo 6](#)). Após a troca do primeiro elemento, é utilizado o procedimento *descer* (ver [Algoritmo 6.2](#) no [Capítulo 6](#)), que reorganiza o heap após a remoção do elemento de maior prioridade.

■

Algoritmo 7.6

Ordenação em heap

```

arranjar( $n$ )
 $m := n$ 
enquanto  $m > 1$  faça
    trocar( $L[1], L[m]$ )
     $m := m - 1$ 
    descer( $1, m$ )
    
```

A complexidade do [Algoritmo 7.6](#) é de fácil avaliação. O procedimento *arranjar* tem complexidade  $O(n)$ ; o procedimento *descer*,  $O(\log n)$ . Como o procedimento *descer* é chamado  $n$  vezes, para cada elemento colocado em sua posição definitiva na tabela, a complexidade total do algoritmo é  $O(n \log n)$ .

A [Figura 7.6](#) apresenta, passo a passo, um exemplo de ordenação da tabela mostrada na [Figura 7.5](#), após o procedimento *arranjar*. Para cada iteração do algoritmo, são ilustrados o efeito na árvore após a troca de nós e o resultado do procedimento *descer*. A última árvore da figura apresenta a tabela já ordenada.

95 60 78 39 28 66 70 33

FIGURA 7.5 Lista de prioridades.

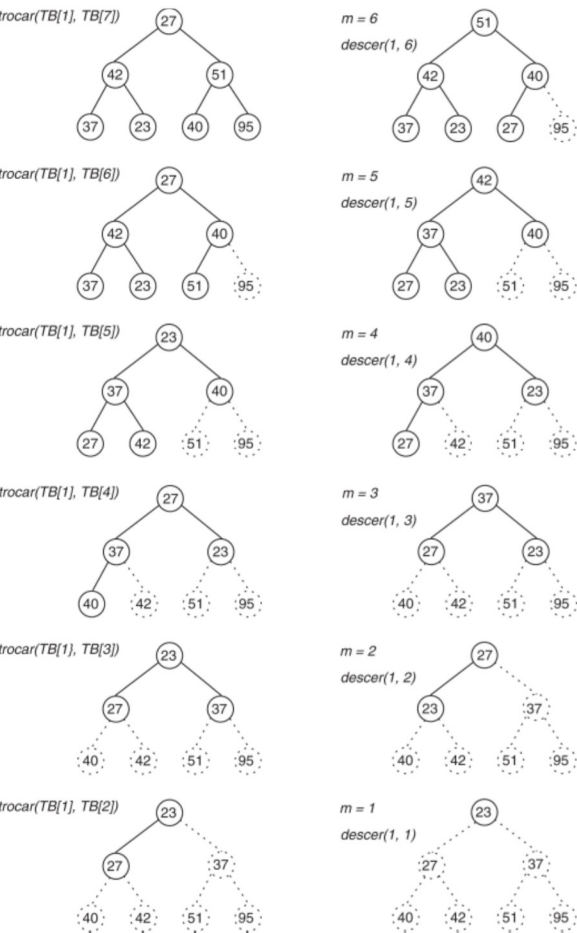


FIGURA 7.6 Ordenação em heap.

## 7.7 Limite Inferior para Algoritmos de Ordenação

Neste capítulo, foram apresentados algoritmos de ordenação com complexidades de pior caso  $O(n^2)$  e  $O(n \log n)$ . Obviamente os que apresentam complexidade  $O(n \log n)$  são melhores, porém para afirmar que um algoritmo é ótimo é necessário conhecer um limite inferior para todos os algoritmos que executam essa tarefa, o que é difícil de ser obtido no caso geral. No caso de algoritmos de ordenação, esse limite inferior é conhecido e essa análise é aqui apresentada. Para tal, é utilizada uma *árvore de decisão*, que representa todos os passos possíveis na execução de um algoritmo. A atenção se concentrará em algoritmos de ordenação por comparações (exatamente os apresentados neste capítulo).

### 7.7.1 Árvore de Decisão

Uma *árvore de decisão* é uma abstração usada para provar limites inferiores. Aqui, será utilizada uma árvore binária. Cada  $n$  representa um conjunto de possíveis ordens de elementos de uma lista, consistente com as comparações entre seus elementos  $j$  realizadas. Para cada  $n$  é colocada uma comparação; as arestas da árvore indicam as duas possibilidades de respostas. A árvore de decisão da [Figura 7.7](#) representa um algoritmo que ordena uma lista de três elementos  $a$ ,  $b$  e  $c$ . O estado inicial do algoritmo se encontra na raiz da árvore, isto é, nenhuma comparação foi feita e todas as ordens são possíveis resultados para a ordenação. O algoritmo considerado nessa árvore é a ordenação bolha. A primeira comparação que este algoritmo efetua é entre  $a$  e  $b$ , isto é, o primeiro elemento e o segundo. Os dois resultados,  $a < b$  e  $b < a$ , levam a dois estados (nós da árvore) distintos. Em cada um deles, as ordens relativas à resposta correta da comparação são encontradas. Quando o algoritmo atinge o nó 2, a comparação é feita entre  $b$  e  $c$ . Se  $b < c$ , o algoritmo chega ao nó 4 e existe somente uma ordem possível; o algoritmo pode terminar, a ordenação está completa. Se  $b > c$ , existem duas ordens possíveis:  $a, c, b$  e  $a, b, c$ . De acordo com o método, neste caso os elementos devem ser trocados:  $c$  passa a ser o segundo elemento e  $b$ , o terceiro. Outra comparação deve ser realizada, novamente entre o primeiro elemento e o segundo, isto é,  $a$  e  $c$ . Em cada alternativa é obtida apenas uma ordem correta. O raciocínio é análogo para os outros nós. Observe que esta árvore corresponde a um determinado algoritmo; entretanto, qualquer algoritmo de ordenação por comparação pode ser representado por uma árvore de decisão. Em qualquer uma delas, o número de comparações efetuadas pelo algoritmo corresponde ao maior comprimento do caminho da raiz até uma de suas folhas.

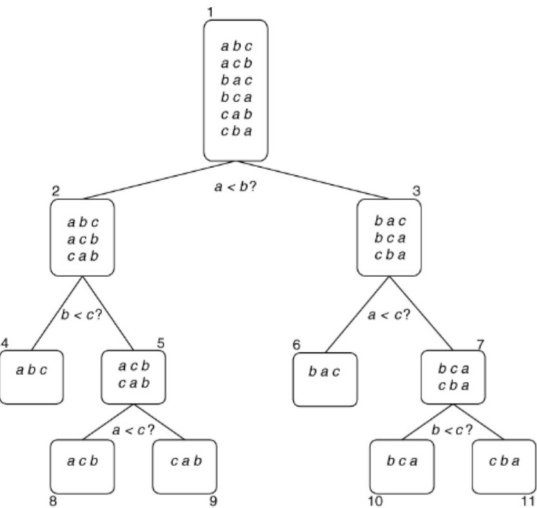


FIGURA 7.7 Árvore de decisão.

### 7.7.2 O Tamanho da Árvore de Decisão

A árvore da [Figura 7.7](#) tem seis folhas, correspondendo às seis possíveis ordenações da lista inicial. Em geral, ao ordenar  $n$  elementos, existem  $n! = n(n - 1) \dots 1$  possíveis saídas, que são as diferentes ordens da lista inicial. Isto significa que qualquer dos  $n$  elementos pode ser o primeiro, qualquer um dos  $n - 1$  restantes pode ser o segundo, e assim sucessivamente. Então, qualquer

árvore de decisão descrevendo um algoritmo de ordenação correto de uma lista de  $n$  elementos precisa ter  $f = n!$  folhas. E mais, o comprimento do caminho da raiz da árvore até uma folha fornece o número de comparações feitas quando a ordenação representada por essa folha é a ordem correta para uma entrada determinada. Então, o comprimento do maior caminho da raiz até uma folha é o limite inferior do número de passos executados pelo algoritmo no pior caso.

Uma árvore binária pode ter, no máximo, 1 nó raiz, 2 nós no nível 2, 4 nós no nível 3, e, em geral,  $2^{i-1}$  no nível  $i$ . Sabe-se que a altura da árvore é igual ao nível máximo de seus nós ([Seção 3.2](#)). Então, o maior número de folhas de uma árvore com altura  $h$  é  $2^{(h-1)}$ . De outra forma, pode-se dizer então que uma árvore binária com  $f$  folhas precisa ter altura, no mínimo,  $O(\log f)$ . Como  $f = n!$ , conclui-se que qualquer algoritmo de ordenação, que somente use comparações para encontrar a ordem correta, precisa  $\Omega(\log(n!))$  no pior caso. Uma boa aproximação para  $n!$  é  $(n/e)^n$ , onde  $e = 2,7183\dots$  é a base de logaritmos neperianos. Como  $\log((n/e)^n) = n \log n - n \log e$ , tem-se que  $\log(n!)$  é da ordem de  $n \log n$ . Um limite inferior mais preciso pode ser obtido ao se observar que  $n(n-1)(n-2)\dots(2)(1)$  é o produto de pelo menos  $n/2$  fatores que valem  $n/2$ . Então,  $n! \geq (n/2)^{(n/2)}$ , e  $\log(n!) \geq (n/2) \log(n/2) = (n/2) \log n - (n/2)$ . Conclui-se então que ordenar por comparações tem complexidade de tempo de  $\Omega(n \log n)$  no pior caso.

## 7.8 Exercícios

### 7.1 Ordenar a lista

10, 4, 7, 15, 12, 26, 6, 9

usando as ordenações bolha e por inserção.

7.2 Analisar o comportamento do [Algoritmo 7.2](#) (ordenação bolha com critério de parada) quanto ao número de trocas e o número de percursos na tabela para as seguintes entradas:

10 20 30 40 50 5

90 10 20 30 40 50

7.3 Qual a complexidade de tempo da ordenação por inserção se todas as chaves são iguais?

7.4 A seguinte tabela foi submetida a um algoritmo de ordenação:

8 7 6 5 4 3 2 1

Em algum ponto da ordenação, a tabela se encontra da seguinte forma:

6 5 3 1 4 2 7 8

Qual o método de ordenação utilizado: bolha, por inserção ou em heap? Justificar sua resposta.

7.5 Estudar o comportamento do [Algoritmo 7.5](#) (ordenação rápida) no caso de a entrada estar ordenada em ordem crescente

ordenada em ordem decrescente

7.6 Mostrar o comportamento da ordenação em heap para a lista

80, 35, 20, 60, 42, 36, 85.

7.7 O que acontece se o [Algoritmo 7.6](#) (ordenação em heap) for utilizado em uma tabela já ordenada? E se a tabela estiver ordenada, porém invertida?

7.8 Assinalar se certo ou errado:

Se a sequência a ser ordenada já se encontrar sob a forma de um heap, o [Algoritmo 7.6](#) requer apenas  $O(n)$  passos para ordenar a sequência.

7.9 O algoritmo que se segue ordena uma tabela  $L$  de  $n$  elementos. Avaliar as complexidades de melhor caso e pior caso do algoritmo. Discutir as estruturas de dados necessárias para a implementação do método.

Passo 1: Construir uma árvore binária de busca com os elementos de  $L$ .

Passo 2: Percorrer a árvore obtida em ordem simétrica.

7.10 Apresentar a árvore de decisão correspondente à ordenação por inserção de uma tabela com três elementos.

7.11 Um algoritmo de ordenação é estável se ele preserva a ordem original dos elementos com chaves iguais. Quais dos métodos apresentados neste capítulo são estáveis?

7.12 Apresentar um algoritmo que receba uma tabela de números inteiros e os ordene em ordem decrescente tendo como critério a sua frequência, isto é, o número de vezes que cada um deles ocorre na tabela.

Exemplo: dada a tabela

1 3 5 6 6 6 3 3 3 1

Resultado:

3 3 3 3 6 6 6 1 1 5.

7.13 Suponha que se deseja ordenar uma lista L consistindo em uma lista ordenada seguida por alguns outros de forma arbitrária. Qual dos métodos descritos neste capítulo melhor se adapta a essa tarefa?

## Notas Bibliográficas

---

Por sua aplicabilidade, métodos de ordenação foram dos primeiros algoritmos a serem discutidos em relação à eficiência. No [Capítulo 7](#) foram apresentados alguns algoritmos clássicos. Os mais simples, a ordenação bolha e por inserção; os de melhor complexidade, o método rápido (proposto por Hoare em 1962 [\[Ho62\]](#)), a ordenação em heap (proposta por Williams em 1964 [\[Wi64\]](#)) e a ordenação por intercalação, mencionada por Knuth [\[Kn73\]](#) como sendo um dos primeiros e mais eficientes métodos propostos. A referência mais tradicional no estudo de ordenação é justamente o livro de Knuth [\[Kn68\]](#), volume 3. Uma revisão sobre a classificação dos métodos de ordenação é vista em [La80a], e Cook e Kim [\[Co80a\]](#) analisam a eficiência dos algoritmos, no caso particular em que a tabela está praticamente ordenada.